

```
for book in books:                # n * steps in the loop
    if 'by Al Sweigart' in book:  # 1 step
        points += 1              # 1 step
return points                      # 1 step
```

The number of steps comes to $1 + (n \times 1) + (n \times 2) + 1$, which becomes $3n + 2$ after combining the like terms. Once we drop the lower orders and coefficients, this becomes $O(n)$, or linear complexity, no matter if we loop through `books` once, twice, or a billion times.

So far, all examples that used a single loop have had linear complexity, but notice that these loops iterated n times. As you'll see in the next example, a loop in your code alone doesn't imply linear complexity, although a loop that iterates over your data does.

This `iLoveBooks()` function prints “I LOVE BOOKS!!!” and “BOOKS ARE GREAT!!!” 10 times:

```
def iLoveBooks(books):
    for i in range(10):                # 10 * steps in the loop
        print('I LOVE BOOKS!!!')      # 1 step
        print('BOOKS ARE GREAT!!!')   # 1 step
```

This function has a `for` loop, but it doesn't loop over the `books` list, and it performs 20 steps no matter what the size of `books` is. We can rewrite this as $20(1)$. After dropping the 20 coefficient, we are left with $O(1)$, or constant time complexity. This makes sense; the function takes the same amount of time to run, no matter what n , the size of the `books` list, is.

Next, we have a `cheerForFavoriteBook()` function that searches through the `books` list to look for a favorite book:

```
def cheerForFavoriteBook(books, favorite):
    for book in books:                # n * steps in the loop
        print(book)                  # 1 step
        if book == favorite:         # 1 step
            for i in range(100):      # 100 * steps in the loop
                print('THIS IS A GREAT BOOK!!!') # 1 step
```
